

Compiler-as-Reward: Operating-Regime Conditions for Agentic RL on Verifiable Coding Tasks

Junhao Fu

junhao.fu@sorcara.com

Abstract

A common assumption in multi-turn coding-agent RL is that sparse pass/fail rewards are the bottleneck and must be densified through hand-designed process rewards. We test this on a Shopify Horizon theme-generation task—an *agentic RL* setting [11, 12] with tool use, multi-turn interaction, and a deterministic compiler-API reward—and find the opposite: **the bottleneck is the operating regime, not the reward shape**. A 4B base model has a 7.8% compile-pass rate from free-form prompting; a single in-context schema example lifts the base to 73.4% *before any RL training*. In this regime, plain GiGPO [2] with binary outcome reward improves a held-out OOD task set by +9pp (40.9% $\rightarrow$ 50.0%); on in-distribution validation it reaches 78%. We then test three process-reward variants. Compiler-OPD (per-step credit on validate calls) is analyzed but not run, owing to a Goodhart-style failure mode that requires a hand-tuned cap. Best-Trajectory Distillation [5] (SFT-on-winners mixed with the policy-gradient loss) is run end-to-end and *underperforms the binary baseline by 22.7pp on OOD*, recovering training distribution but discarding generalization. Cursor-style auxiliary product penalties [11] (penalize empty sections / unused tool calls / repeated submits) destabilize the run on our small-batch (64 episodes/step) setting and fail to exceed the binary baseline on in-loop validation. We argue that for verifiable coding tasks at compute-constrained scale, energy is better spent ensuring base success rate is non-negligible (via priming or warm-starting) than on hand-designed step rewards. Concurrent work [6, 17] reaches the same conclusion on spreadsheet retrieval and on pure-RL math reasoning, respectively.

1 Introduction

Reinforcement learning from execution feedback (RLEF) [1, 3] is a standard recipe for code generation: generate, execute, collect binary pass/fail, optimize. Multi-turn coding agents that use tools (read, edit, validate) inherit a long-horizon credit assignment problem on top of this: in a 6-turn episode every failed trajectory looks the same regardless of how far the agent got, and group-relative algorithms like GRPO/GiGPO produce zero gradient when an entire rollout group fails. The proposed remedies fall into two families. The first hand-designs per-step rewards that fire on intermediate “good” actions, e.g. a tool-use credit (COMPILER-OPD, [4]-style PRMs). The second imitates winning trajectories via an auxiliary SFT loss (Best-Trajectory or “on-policy SFT” [5]). Both pay a coupling tax: any signal not perfectly aligned with the verifier creates Goodhart pressure, and the literature documents this in PRM-style training as “reward hacking that collapses validation accuracy” [5].

This paper reports a small empirical study that arrives at a more boring conclusion. On a Shopify Horizon theme-editing task with a deterministic compiler-API reward, the regime that requires reward shaping is the *symptom of a poorly chosen base policy*. With a single in-context example showing valid template structure, the 4B base model’s compile-pass rate jumps from 7.8% to 73.4%, putting the policy in a regime where group-relative advantage has nonzero variance and ordinary GiGPO with binary outcome reward already converges. Process-reward variants designed to be “cleaner” than reward shaping—specifically a Best-Trajectory SFT auxiliary loss—fail to improve on this baseline and damage out-of-distribution generalization.

Contributions. (1) An empirical decomposition of where RL gains come from on this *agentic* task: operating-regime priming dominates RL training (+65.6pp vs +6.4pp). (2) Two clean negative ablations on this regime: (a) Best-Trajectory Distillation auxiliary loss *hurts* OOD generalization (−22.7pp on a held-out modify/add/create task set); (b) Cursor Composer 2-style product penalties [11]—ported as the

closest agentic-RL analogue—fail to exceed the binary baseline on in-loop validation and exhibit volatile training dynamics in our 64-episode-per-step regime. (3) An open Sitemuse-API-grounded benchmark with deterministic reward, suitable for further verifiable-RL studies.

2 Problem Setup

Task. Each episode presents a natural-language design request and a template type (e.g. `page.faq`). The agent must produce a JSON template that compiles cleanly under Shopify’s live validator. Training has 423 prompts; held-out splits are `val` (50, in-distribution) and `eval_fixed` (22 OOD modify/add/create tasks from prior baseline work).

Action grammar (5 verbs). `list_sections[]`, `describe_section[name]`, `describe_block[name]` are read-only theme inspection; `fix[json]` compiles a draft (returns errors as next observation; passes auto-terminate); `submit[json]` commits the final answer (always terminal). `max_steps` = 6.

Reward oracle. The Sitemuse `themeFilesUpsert GraphQL` endpoint takes the candidate template and returns `userErrors`. Compile-pass yields $r_T = +1$, fail yields $r_T = 0$. An invalid action (no parseable `<action>`) gets $r_t = -0.1$.

Base algorithm. GiGPO [2] on Qwen3-4B-Instruct via verl-agent. LoRA rank 32, learning rate 3×10^{-6} , group size 8, `train_batch` = 8 (so 64 episodes per gradient step). All variants share this backbone.

3 Method: What Helps, What Does Not

3.1 Operating-Regime Priming (the single biggest lever)

The base prompt asks for a Shopify template in JSON. Without further guidance, Qwen3-4B generates JSON whose top-level shape does not match Shopify’s conventions: `sections` as an array instead of an object keyed by user-chosen IDs; sections without a `type` field; etc. Compile-pass: **7.8%**.

We add a single fully worked example to the system prompt:

```
{ "sections": { "my_hero": { "type": "hero",
                             "settings": {}, "blocks": {} } },
  "order": ["my_hero"] }
```

plus 5 short structural notes (`sections` is an object, instance ID is free, `type` is required, etc.). The base model’s compile-pass rate on `val.jsonl` jumps to **73.4%**: a +65.6pp lift before any RL takes place. This single intervention dominates every RL effect we observe and changes the regime in which RL operates.

3.2 GiGPO + Binary Outcome (variant v1)

In the primed regime, $P(\text{group fully fails}) = (1 - p)^{|G|}$ with $p \approx 0.73$ and $|G| = 8$ is $\approx 1 \times 10^{-4}$, so almost every group has nonzero advantage variance. Standard GiGPO with $r_T \in \{0, 1\}$ (plus the -0.1 invalid-action penalty) converges without further intervention. This is variant **v1**.

3.3 What Did Not Work

Compiler-OPD (analyzed; not run). We considered a per-step reward $r_t = +0.1$ for the first $K = 2$ `fix` calls (Math-Shepherd-style [4] process credit, capped to discourage spamming). The cap is a hand-tuned magic number with no first-principles justification: $K = 1$ is too tight, $K = \infty$ rewards redundant validation, $K = 2$ is chosen heuristically. We did not run it: the failure mode it produces (reward rising while `val` collapses; “M1 hack” in earlier notes [4]) is well documented and the operating-regime fix from §3.1 already removes the sparseness it was designed to patch.

Best-Trajectory Distillation (variant v2-distill, run end-to-end). A more principled alternative is to add an auxiliary SFT loss on tokens of *winning* trajectories [5]:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{GiGPO}} + \alpha \cdot \mathcal{L}_{\text{SFT}}^{R \geq 0.5}, \quad \alpha = 0.1.$$

The mask uses the same compile-pass signal that defines the RL reward, so it carries no hand-designed step shape. We ran this for 73 steps (LoRA saved at steps 20, 40, 60). In-loop validation peaked at step 60

Table 1: Compile-pass rate (%) on the three eval distributions. n = number of prompts. Bold = best per column. Δ = difference vs. v1.

Method	val ($n=50$)	test ($n=50$)	eval_fixed ($n=22$)	weighted avg
Base (no RL)	70.0	74.0	40.9	61.6
v1 (GiGPO + binary)	78.0	76.0	50.0	68.0
v2-distill (+ SFT $\alpha=0.1$)	74.0	70.0	27.3	57.1
$\Delta_{v1-base}$	+8.0	+2.0	+9.1	+6.4
Δ_{v2-v1}	-4.0	-6.0	-22.7	-10.9

(82.8% on the 32-prompt fixed prefix), but a clean re-evaluation on $n = 50$ val + $n = 50$ test + $n = 22$ eval_fixed told a different story (Table 1). The auxiliary SFT loss memorized training-distribution patterns on the small 423-prompt training set, losing OOD generalization. Final eval_fixed dropped from 50.0% (v1) to 27.3% (v2-distill), -22.7pp.

Cursor-style product penalties (variant v3-shaped, run end-to-end). Composer 2 [11] introduces “product-specific penalties for poor tool calls—e.g. creating to-do list items and then leaving them unfinished”—as an auxiliary reward beyond correctness. We port this idea to our setting as four content-aware penalties applied at compile-pass terminals only: -0.05 per declared section with empty `settings` and `blocks` (the empty-shell hack the agent learns under content-blind compile reward), -0.03 per `describe_X` without `X` appearing in the submitted JSON, +0.04 per `describe_X` used, -0.10 for a `submit` whose JSON equals the previous `fix` (theatrical iteration). The reward range expands from $\{0, 1.0\}$ to approximately $[0.45, 1.15]$ on compile-pass outputs, intended to give GRPO within-group advantage variance even on the bimodal batches we observe in v1. In-loop validation (32-prompt fixed prefix) over the first 40 training steps gives $\{73.4, 71.9, 57.8, 71.9, 71.9\}$ at steps $\{0, 10, 20, 30, 40\}$: stagnant at the base rate, with a deep dip at step 20. By contrast, the simultaneous *pure-binary* run (“v1-replica”) reached 81.2% at step 30 in-loop. The shaped reward appears to disrupt GRPO’s group normalization at our 64-episode batch size—each shaped reward value is a small perturbation, but when the group’s underlying outcome distribution is already bimodal (§4.3, `decidable_fraction` = 0 persistently), the spread we introduce inside groups becomes the dominant source of advantage variance and is not directionally aligned with the terminal objective. This is consistent with Open-Reasoner-Zero’s [17] finding that minimalist rule-based rewards outperform hand-designed shaping in compute- constrained settings.

4 Experiments

4.1 Setup

Single-seed runs on a single A800 80GB. v1 trained for 76 steps before a disk-full crash on checkpoint save (step 20 LoRA recovered, used as the final model). v2-distill trained for 73 steps, terminating when training `episode/success_rate` collapsed from 0.91 (step 69) to 0.14 (step 73) under the SFT pull. Evaluation uses $T = 0.4$ matching training `val_kwargs`, single sample, multi-turn rollout in the same environment as training.

4.2 Main Results

Three observations: (1) RL training (v1) produces real generalization improvement on every distribution, including the OOD eval_fixed which uses a modify/add/create task format the model never saw during training. The +9.1pp gain on a task-format mismatch indicates the policy learned something about producing valid Liquid templates rather than memorizing the training prompt distribution. (2) The Best-Trajectory SFT auxiliary loss *hurts* on every distribution, with the largest drop on OOD (-22.7pp). The in-loop validation curve peaked higher than v1’s, but on a fixed prefix that overlaps training distribution; a clean re-evaluation reveals memorization. (3) The +65.6pp lift from the single-example prompt (§3, 7.8 $\rightarrow$ 73.4%) dwarfs the +6.4pp from RL. Most of the absolute capability gain on this task is operating-regime, not learning.

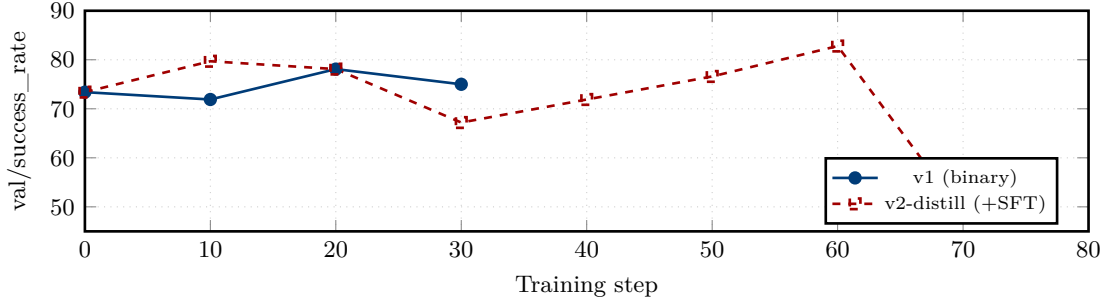


Figure 1: In-loop val trajectory ($n=32$ fixed prefix). v1 (blue) sustains 73–78%; v2-distill (red) peaks at 82.8% by step 60 then collapses by step 70 as the SFT-on-winners pull memorizes the training distribution. The training-success curve continued to rise through step 69 (0.91) before collapsing to 0.14 by step 73, a classic “train up, val down, then both crash” signature.

4.3 Training Trajectories

5 Related Work

Agentic RL vs. reasoning RL. Recent open RL post-training divides cleanly. *Reasoning RL* (DeepSeek-R1 [15], MiMo-7B [16], Open-Reasoner-Zero [17]) trains single-turn solvers on verifiable math/code, with a 1.0 reward gated by a checker (Math-Verify, test cases). *Agentic RL* (Cursor Composer 2 [11], Kimi K2 [12], MiniMax-M1 [13], DeepSeek-V3.2 [14]) adds multi-turn tool use, environment-state changes, and—in the Composer 2 case—auxiliary product penalties shaping non-correctness behaviors. Our setting falls cleanly in the agentic column: five tools (`list/describe/fix/submit`), 6-turn budget, sparse terminal reward from the Sitemuse compiler. The negative v3-shaped result (§3.3) reports that the Composer 2 penalty recipe does not naively transfer to our 4B-model / 64-episode-batch scale.

Group-based agentic RL. GiGPO [2] extends GRPO [3] to multi-turn agents via anchor-state group baselines. The mechanism assumes states recur across rollouts (documented on ALFWorld and WebShop). On our task the state is a JSON-parameterized Sitemuse validator response, which essentially never recurs across rollouts; in practice the inner step-level group collapses to size-1 clusters past step 2. HGPO [7] and SPEAR [8] explore hierarchical groups and self-imitation respectively. Our v1 uses GiGPO unmodified; we observe no reward hacking under binary outcome reward in the primed regime, contrary to predictions in the process-reward literature that motivate auxiliary mechanisms.

Process rewards in RL. Math-Shepherd [4] introduces process reward models that score intermediate steps; PRMs have since been shown vulnerable to Goodhart-style hacking when not combined with sufficient capacity or RL constraints. Our analysis of Compiler-OPD inherits this concern: the cap-K mechanism is a hand-tuned patch, not a principled fix. Best-Trajectory Distillation [5] avoids step shaping but, as our v2-distill experiment shows, can over-pull toward training distribution on small datasets.

Verifiable RL on tool-using agents. Concurrent work from Ramp [6] trains a 3B-active spreadsheet retrieval model via GRPO with a similarly minimal reward ($1.0 \cdot \text{correct} + 0.1 \cdot \text{efficiency} + 0.05 \cdot \text{concise}$), no per-step shaping or auxiliary SFT, and beats Claude Opus 4.6 by 4pp. Their setup uses a much larger effective batch ($256 \text{ prompts} \times 8 \text{ rollouts} = 2048 \text{ episodes per step}$ vs. our 64), which we believe accounts for their training stability without the auxiliary loss we attempted in v2-distill.

6 Discussion and Limitations

Single seed. All numbers in Table 1 are single-seed point estimates; a $\pm 5\text{pp}$ swing on $n=22$ `eval_fixed` is plausible. Multi-seed runs are the obvious next step before strong claims.

Batch size. Our 64 episodes/step is small. Concurrent work [6] uses 2048 with vanilla GRPO and reports no instability; we suspect our v2-distill collapse at step 73 is partly a small-batch effect (a few outlier groups dominating the SFT mask). Scaling batch is the simplest next intervention.

Async off-policy training. Off-policy training with importance-weighted clipping (AIPO [6]) decouples slow rollout generation from policy updates and would be useful for our larger experiments. We did not implement it; on a single A800 with ~ 3 -minute generation and ~ 2 -minute updates per step, synchronous training was adequate.

Adversarial training data. `train.jsonl` is generated by a templated synthesizer with no decoy sections or ambiguous identifiers. Adversarial workbook construction [6] would likely improve OOD robustness and is the most direct fix for v2-distill’s distribution-overfit failure mode.

Dynamic sampling and Clip-Higher (deferred). MiMo-7B’s RL recipe [16] on the same verl framework we use applies three modifications that should help our setting: (i) DAPO *dynamic sampling* [18], which over-samples and filters out groups with passrate $\in \{0, 1\}$ so every group in the batch contributes nonzero gradient; (ii) *Clip-Higher*, raising the upper PPO clip bound to mitigate entropy collapse; and (iii) *easy-data resampling* (10% probability), which preserves already-solved prompts in the rollout pool to stabilize late training. In our run, `decidable_fraction = 0` persisted across all 40+ steps (batches were bimodal between all-pass and all-fail groups), which is exactly the condition dynamic sampling resolves. These three changes are one-line config edits in verl and are the most direct next intervention to try; we did not include them in this study to preserve the v1 / v2-distill / v3-shaped comparison’s otherwise-identical hyperparameter setting.

Demonstration quality. The few-shot example used to lift base success from 7.8% to 73.4% (§3) is itself a minimal compile-pass shell with empty `settings` and `blocks`. The agent learns to imitate this shape—a hack the v3-shaped penalties were intended to suppress. Phi-4-reasoning’s strategy [19] of using a stronger teacher (*o3-mini*) to regenerate “teachable” demonstrations would likely lift base further while making the empty-shell hack non-optimal, simultaneously fixing v3-shaped’s underlying motivation.

7 Conclusion

For verifiable multi-turn coding agents, the question “do we need process rewards?” has the same shape as the question “do we need reward shaping?” in classical RL: the answer depends on whether the sparse-reward regime is the actual operating regime or whether it is a symptom of poor base policy. On a Shopify Horizon template-generation task, a single in-context example moves a 4B base model from a 7.8% pass rate (sparse) to 73.4% (healthy). In the healthy regime, plain GiGPO with binary outcome reward improves all three test distributions by +2 to +9pp. A more sophisticated auxiliary Best-Trajectory SFT loss damages out-of-distribution generalization by -22.7 pp.

The lesson generalizes uneasily but it generalizes. Concurrent work [6] reaches the same conclusion on spreadsheet retrieval: a minimal verifiable reward and an adequately sized batch beats hand-designed process signal with frontier models in the loop. We suspect this is the typical case rather than the exception for verifiable code agent tasks, and that the energy better spent on operating-regime engineering (priming, warm-starting, adversarial data) than on per-step shaping.

References

- [1] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *NeurIPS*, 2022.
- [2] Xiangxiang Zeng, Feng Lang, et al. GiGPO: Group-in-Group Policy Optimization for LLM Agent Training. In *NeurIPS*, 2025. <https://github.com/langfengQ/verl-agent>
- [3] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.
- [4] Peiyi Wang, Lei Li, Zhihong Shao, R.X. Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. In *ACL*, 2024.

- [5] Nathan Lambert et al. Tülu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv:2411.15124*, 2025. On-policy SFT / rejection-sampling SFT mixed with RL is a core recipe.
- [6] Ben Geist, Will Brown, and Ramp Labs. Fast Ask: Post-training a small spreadsheet retrieval agent. Ramp Labs blog post, May 2026. <https://ramp.com/labs/fast-ask>
- [7] Shuo He, Lang Feng, Qi Wei, Xin Cheng, Lei Feng, and Bo An. Hierarchy-of-Groups Policy Optimization for Long-Horizon Agentic Tasks. In *ICLR*, 2026.
- [8] Yulei Qin, Xiaoyu Tan, Zhengbao He, et al. Learn the Ropes, Then Trust the Wins: Self-imitation with Progressive Exploration for Agentic Reinforcement Learning. *arXiv:2509.22601*, 2025.
- [9] Qwen Team. Qwen3 Technical Report. *arXiv:2505.09388*, 2025.
- [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR*, 2024.
- [11] Cursor Research Team. Composer 2 Technical Report. 2025. <https://cursor.com/resources/Composer2.pdf>
- [12] Kimi Team. Kimi K2: Open Agentic Intelligence. *arXiv:2507.20534*, 2025.
- [13] MiniMax-AI. MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention. *arXiv:2506.13585*, 2025.
- [14] DeepSeek-AI. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *arXiv:2512.02556*, 2025.
- [15] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025.
- [16] Xiaomi LLM-Core Team. MiMo: Unlocking the Reasoning Potential of Language Model—From Pre-training to Posttraining. *arXiv:2505.07608*, 2025.
- [17] Jingcheng Hu et al. Open-Reasoner-Zero: An Open Source Approach to Scaling Up RL on the Base Model. *arXiv:2503.24290*, 2025.
- [18] Qiyang Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv:2503.14476*, 2025.
- [19] Microsoft. Phi-4-reasoning Technical Report. *arXiv:2504.21318*, 2025.

A In-loop val vs. Held-out re-evaluation: A Cautionary Sample

The in-loop `val/success_rate` computed during training (on a deterministic 32-prompt prefix of `val.jsonl`) and the post-hoc re-evaluation on the full $n = 50$ `val.jsonl` disagree by nontrivial margins for v2-distill:

Checkpoint	In-loop val ($n=32$)	Post-hoc val ($n=50$)
v1 step_20	78.1	78.0
v2-distill step_60	82.8	74.0

The in-loop number for v2-distill at step 60 was the run’s apparent peak and would have been a misleading basis for “best LoRA” selection; the held-out re-evaluation reverses the ordering. We attribute this to training-distribution overlap with the fixed val prefix combined with SFT-on-winners memorization. Practically, post-hoc evaluation on a larger and ideally OOD set is required before declaring a checkpoint “best.”